Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: https://github.com/Ashwath-Raj/CSE-204-DAA-assignments-

1. Sort Array Elements Using Merge Sort Technique

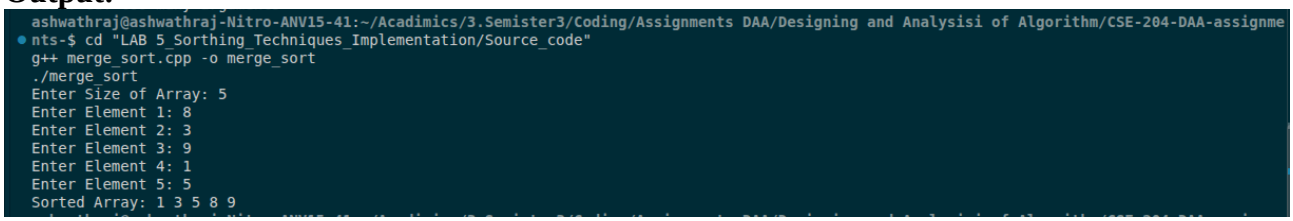
Problem

Write a program to sort the array elements using Merge Sort Technique. Derive its time complexity.

Method

Merge Sort divides the array into two halves recursively, sorts each half and merges the sorted halves to obtain the final sorted array.

Output:



Code:

```
/*
Program Name: Merge Sort
Program Description: Takes an array as input and sorts its elements using
                     the Merge Sort technique
Program Creation Time: Sep 6th
Program Creator: Ashwath Raj | AP25110010712
*/

#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right)
{
    int leftSize = mid - left + 1;
    int rightSize = right - mid;
    int *leftArr = new int[leftSize];
    int *rightArr = new int[rightSize];

    for (int i = 0; i < leftSize; i++)
        leftArr[i] = arr[left + i];

    for (int j = 0; j < rightSize; j++)
        rightArr[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < leftSize && j < rightSize)
    {
        if (leftArr[i] <= rightArr[j])
            arr[k++] = leftArr[i++];
        else
            arr[k++] = rightArr[j++];
    }

    while (i < leftSize)
        arr[k++] = leftArr[i++];

    while (j < rightSize)
        arr[k++] = rightArr[j++];

    delete[] leftArr;
    delete[] rightArr;
}

void mergeSort(int arr[], int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
```

```
}

int main(void)
{
    int n;

    cout << "Enter Size of Array: ";
    cin >> n;

    int *arr = new int[n];

    for (int i = 0; i < n; i++)
    {
        cout << "Enter Element " << i + 1 << ": ";
        cin >> arr[i];
    }

    mergeSort(arr, 0, n - 1);

    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;

    delete[] arr;

    return 0;
}
```

Complexity

Time Complexity: O(n log n)
Space Complexity: O(n)

Time Complexity Derivation

At each level, merging takes O(n). Thus T(n) = 2T(n/2) + O(n). There are log n levels, so T(n) = O(n log n).

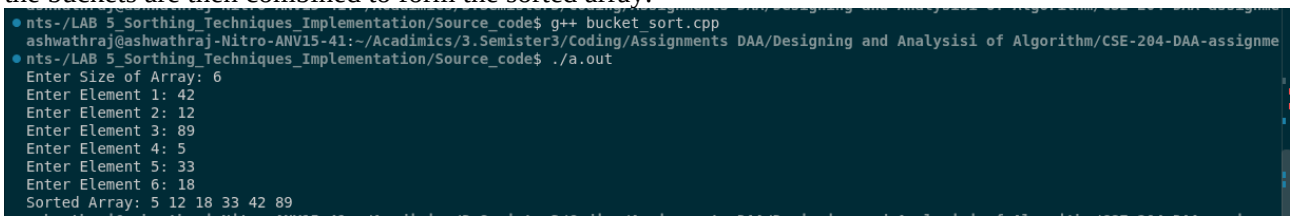
2. Sort Array Elements Using Bucket Sort Technique

Problem

Write a program to sort the array elements using Bucket Sort Technique. Derive its time complexity.

Method

The elements are distributed into buckets according to their values. Each bucket is sorted individually and the buckets are then combined to form the sorted array.



Output:

Code:

```
/*
Program Name: Bucket Sort
Program Description: Takes an array as input and sorts its elements using
                     the Bucket Sort technique
Program Creation Time: Sep 6th
Program Creator: Ashwath Raj | AP25110010712
*/

#include <iostream>
#include <vector>
using namespace std;

void sortBucket(vector<int> &bucket)
{
    for (int i = 1; i < static_cast<int>(bucket.size()); i++)
    {
        int value = bucket[i];
        int j = i - 1;
```

```
        while (j >= 0 && bucket[j] > value)
        {
            bucket[j + 1] = bucket[j];
            j--;
        }

        bucket[j + 1] = value;
    }
}

void bucketSort(int arr[], int n)
{
    if (n <= 1)
        return;

    int minValue = arr[0];
    int maxValue = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] < minValue)
            minValue = arr[i];
        if (arr[i] > maxValue)
            maxValue = arr[i];
    }

    long long range = static_cast<long long>(maxValue) - minValue + 1;
    vector<vector<int>> buckets(n);

    for (int i = 0; i < n; i++)
    {
        int bucketIndex = static_cast<int>(
            (static_cast<long long>(arr[i]) - minValue) * n / range);
        buckets[bucketIndex].push_back(arr[i]);
    }

    for (vector<int> &bucket : buckets)
        sortBucket(bucket);

    int index = 0;
    for (const vector<int> &bucket : buckets)
    {
        for (int value : bucket)
            arr[index++] = value;
    }
}

int main(void)
{
    int n;

    cout << "Enter Size of Array: ";
    cin >> n;

    int *arr = new int[n];

    for (int i = 0; i < n; i++)
    {
        cout << "Enter Element " << i + 1 << ": ";
        cin >> arr[i];
    }

    bucketSort(arr, n);

    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;

    delete[] arr;

    return 0;
}
```

Complexity

Average Time Complexity: O(n)
Worst Case Time Complexity: $O(n^2)$

Space Complexity: O(n)

Time Complexity Derivation

Distribution and bucket traversal take O(n). If bucket i contains k_i elements, sorting costs $O(\Sigma k_i^2)$. With uniform distribution this is O(n); if all elements fall into one bucket, it becomes $O(n^2)$.

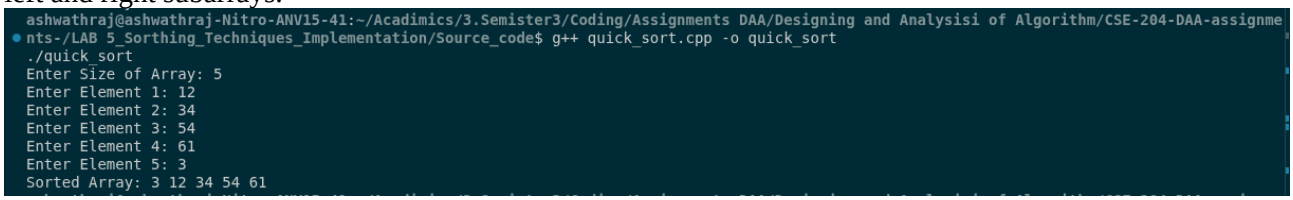
3. Sort Array Elements Using Quick Sort Technique

Problem

Write a program to sort the array elements using Quick Sort Technique. Derive its time complexity.

Method

Quick Sort selects the last element as the pivot, partitions the array around the pivot and recursively sorts the left and right subarrays.



Output:

Code:

```
/*
Program Name: Quick Sort
Program Description: Takes an array as input and sorts its elements using
                     the Quick Sort technique
Program Creation Time: Sep 6th
Program Creator: Ashwath Raj | AP25110010712
*/

#include <iostream>
using namespace std;

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int smallerIndex = low - 1;

    for (int currentIndex = low; currentIndex < high; currentIndex++)
    {
        if (arr[currentIndex] <= pivot)
        {
            smallerIndex++;
            swap(arr[smallerIndex], arr[currentIndex]);
        }
    }

    swap(arr[smallerIndex + 1], arr[high]);
    return smallerIndex + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low >= high)
        return;

    int pivotIndex = partition(arr, low, high);

    quickSort(arr, low, pivotIndex - 1);
    quickSort(arr, pivotIndex + 1, high);
}

int main(void)
{
    int n;

    cout << "Enter Size of Array: ";
    cin >> n;

    int *arr = new int[n];

    for (int i = 0; i < n; i++)
    {
```

```
        cout << "Enter Element " << i + 1 << ": ";
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;

    delete[] arr;

    return 0;
}
```

Complexity

Average Time Complexity: O(n log n)
Worst Case Time Complexity: $O(n^2)$
Space Complexity: O(log n) average, O(n) worst case

Time Complexity Derivation

Partitioning takes O(n). For balanced partitions, $T(n) = 2T(n/2) + O(n) = O(n \log n)$. For repeatedly unbalanced partitions, $T(n) = T(n-1) + O(n) = O(n^2)$.